

✓ Tarea 01

A.1

Realizado por:

Samuel Huertas Rojas

```
1 # --- Librerías a importar ---
2 # %load_ext autoreload
3 # %autoreload 2
4 # import sys
5
6 # sys.path.append("../Tarea_01")
7
8 import numpy as np
9 import matplotlib.pyplot as plt
10 from abc import ABC, abstractmethod
11
12 from matplotlib.animation import FuncAnimation, PillowWriter
13
```

```
1 # Algoritmos para la factorización de matrices y sustitución progresiva y regresiva
2 # ----- Factorización LU -----
3 def matrix_factorization_nxn(
4     diagonal: np.array, # Vector que contiene los coeficientes de la matriz (diagonal)
5     diagonal_superior: np.array, # Vector que contiene los coeficientes de la matriz (diagonal superior)
6     diagonal_inferior: np.array, # Vector que contiene los coeficientes de la matriz (diagonal inferior)
7 ):
8     """
9     Función para realizar la factorización de una matriz tridiagonal  $A = L*U$ 
10     donde L es una matriz triangular inferior con 1's en la diagonal principal y
11     U es una matriz triangular superior.
12     Parámetros:
13     diagonal : np.array
14         Vector que contiene los coeficientes de la matriz (diagonal)
15     diagonal_superior : np.array
16         Vector que contiene los coeficientes de la matriz (diagonal superior)
17     diagonal_inferior : np.array
18         Vector que contiene los coeficientes de la matriz (diagonal inferior)
19     Retorna:
20     l_diag : np.array
21         Vector que contiene los coeficientes de la matriz L (diagonal)
22     l_diag_inf : np.array
23         Vector que contiene los coeficientes de la matriz L (diagonal inferior)
24     u_diag_sup : np.array
25         Vector que contiene los coeficientes de la matriz U (diagonal)
26     """
27     l_diag = np.zeros(len(diagonal))
28     l_diag_inf = np.zeros(len(diagonal_inferior))
29     u_diag_sup = np.zeros(len(diagonal_superior))
30
31     l_diag[0] = diagonal[0]
32     l_diag_inf = diagonal_inferior.copy()
33     for i in range(0, len(diagonal_superior)):
34         u_diag_sup[i] = diagonal_superior[i] / l_diag[i]
35         l_diag[i + 1] = diagonal[i + 1] - l_diag_inf[i] * u_diag_sup[i]
36
37     return l_diag, l_diag_inf, u_diag_sup
38
39
40 # ----- Algoritmos para la sustitución progresiva y regresiva -----
41 # === Sustitución Regresiva ===
42 # Solución del sistema  $Ux = z$ 
43 def sustitucion_regresiva(
44     u_diagonal_superior: np.array, # Vector que contiene los coeficientes de la matriz U (diagonal superior)
45     termino_independiente: np.array, # Vector que contiene los términos independientes
46 ):
47     x = np.zeros(len(termino_independiente))
48     x[-1] = termino_independiente[-1]
49     for i in range(len(termino_independiente) - 2, -1, -1): # de n-2 hasta 0
50         x[i] = termino_independiente[i] - u_diagonal_superior[i] * x[i + 1]
51
52     return x
53
54
55 # === Sustitución Progresiva ===
56 # Solución del sistema  $Lz = b$ 
57 def sustitucion_progresiva(
58     l_diagonal: np.array, # Vector que contiene los coeficientes de la matriz L (diagonal)
59     l_diagonal_inferior: np.array, # Vector que contiene los coeficientes de la matriz L (diagonal inferior)
60     termino_independiente: np.array, # Vector que contiene los términos independientes
61 ):
```

```

61 ):
62     z = np.zeros(len(termino_independiente))
63     z[0] = termino_independiente[0] / l_diagonal[0]
64     for i in range(1, len(termino_independiente)):
65         z[i] = (
66             termino_independiente[i] - l_diagonal_inferior[i - 1] * z[i - 1]
67         ) / l_diagonal[i]
68
69     return z

```

```

1 class Esquema(ABC):
2     def __init__(self):
3         pass
4
5     @abstractmethod
6     def calcular_esquema(self):
7         pass

```

```

1 # --- Función para animar y guardar la animación ---
2 def animar_esquema(
3     N,
4     condicion_inicial,
5     esquema: Esquema,
6     filename: str,
7 ):
8     """
9     Genera y guarda una animación de la evolución de V usando esquema_prog_prog.
10
11     Parámetros
12     -----
13     N : int
14         Número de pasos de tiempo.
15     diag : array
16         Diagonal principal.
17     diag_sup : array
18         Diagonal superior.
19     F : array
20         Término fuente.
21     condicion_inicial : array
22         Condición inicial.
23     filename : str
24         Nombre del archivo de salida (GIF).
25     """
26     # Matriz para guardar los resultados
27     V = esquema.calcular_esquema()
28
29     # --- Animación ---
30     fig, ax = plt.subplots()
31     (line,) = ax.plot(V[0, :], lw=2)
32     ax.set_ylim(np.min(V), np.max(V) + 0.5)
33     ax.set_xlim(0, len(condicion_inicial) - 1)
34     ax.set_title(filename)
35     ax.set_xlabel("Espacio")
36     ax.set_ylabel("$u(t,x)$")
37     ax.grid(True)
38
39     def update(frame):
40         line.set_ydata(V[frame, :])
41         return (line,)
42
43     ani = FuncAnimation(fig, update, frames=range(N + 1), interval=100, blit=True)
44
45     # Guardar como GIF
46     gif_name = filename + ".gif"
47     ani.save(gif_name, writer=PillowWriter(fps=10000))
48     plt.close(fig)
49     print(f"✅ Animación guardada en {filename}")
50
51     return ani

```

✓ Ejercicio 1. 40/50



Para tener una mejor comparación entre la solución exacta y la predicha por los esquemas, se propone utilizar la Raíz del Error Cuadrático Medio (RMSE), por sus siglas en inglés Root Mean Square Derivation. Esta medida nos permite conocer y medir la diferencia entre el valor observado o "verdadero" con el valor predicho. La fórmula matemática es:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{x}_i - x_i)^2}$$

Donde:

- n : es el numero de observaciones.
- $\hat{x}_i$: son los valores predichos.
- x_i : son los valores verdaderos o observados.

```
1 def RMSD(V_verdadero, V_aproximado):
2     suma = 0
3     for i in range(V_verdadero.shape[0]):
4         suma += (V_verdadero[i] - V_aproximado[i]) ** 2
5     rmsd = np.sqrt(suma / V_verdadero.size)
6
7     return rmsd
```

El valor del RMSD nos puede decir que tan similares son la aproximación numérica con el valor verdadero de la función.

- Un valor bajo indica una alta similitud entre los valores predichos por el esquema y los valores verdaderos.
- Un alto valor sugiere una mayor diferencia entre los valores predichos por el esquema y los valores verdaderos.

```
1 # Variables de la parte espacial
2 a = 0 # Limite inferior del intervalo a evaluar
3 b = 1.2 # Limite superior del intervalo a evaluar
4 M = 200 # Número de divisiones de la parte espacial
5
6 h = (b - a) / M # Tamaño de paso: h = (b-a)/M
7
8 # Variables de la parte temporal
9 T = 0.8 # Limite superior del intervalo temporal
10 N = 2000 # Número de divisiones de la parte temporal
11
12 k = T / N # Tamaño de paso: k = T/N
13
14 variable_temporal = np.arange(0, T + k, k)
15 variable_espacial = np.arange(a, b, h)
16
17 t_comparación = [0.2, 0.4, 0.6, 0.8] # Valores de tiempo para realizar comparación
18
19 alpha = 1 / 2 # Constante que acompaña a la derivada espacial
20
21 print(f"Valores a evaluar temporalmente: {variable_temporal}")
22 print(f"Valores a evaluar espacialmente: {variable_espacial}")
23 print(f"Valores de las constantes h: {h} y k:{k}, k/h: {k / h}")
```

```
Valores a evaluar temporalmente: [0.000e+00 4.000e-04 8.000e-04 ... 7.992e-01 7.996e-01 8.000e-01]
Valores a evaluar espacialmente: [0.    0.006 0.012 0.018 0.024 0.03   0.036 0.042 0.048 0.054 0.06   0.066
0.072 0.078 0.084 0.09   0.096 0.102 0.108 0.114 0.12   0.126 0.132 0.138
0.144 0.15   0.156 0.162 0.168 0.174 0.18   0.186 0.192 0.198 0.204 0.21
0.216 0.222 0.228 0.234 0.24   0.246 0.252 0.258 0.264 0.27   0.276 0.282
0.288 0.294 0.3   0.306 0.312 0.318 0.324 0.33   0.336 0.342 0.348 0.354
0.36   0.366 0.372 0.378 0.384 0.39   0.396 0.402 0.408 0.414 0.42   0.426
0.432 0.438 0.444 0.45   0.456 0.462 0.468 0.474 0.48   0.486 0.492 0.498
0.504 0.51   0.516 0.522 0.528 0.534 0.54   0.546 0.552 0.558 0.564 0.57
0.576 0.582 0.588 0.594 0.6   0.606 0.612 0.618 0.624 0.63   0.636 0.642
0.648 0.654 0.66   0.666 0.672 0.678 0.684 0.69   0.696 0.702 0.708 0.714
0.72   0.726 0.732 0.738 0.744 0.75   0.756 0.762 0.768 0.774 0.78   0.786
0.792 0.798 0.804 0.81   0.816 0.822 0.828 0.834 0.84   0.846 0.852 0.858
0.864 0.87   0.876 0.882 0.888 0.894 0.9   0.906 0.912 0.918 0.924 0.93
0.936 0.942 0.948 0.954 0.96   0.966 0.972 0.978 0.984 0.99   0.996 1.002
1.008 1.014 1.02   1.026 1.032 1.038 1.044 1.05   1.056 1.062 1.068 1.074
1.08   1.086 1.092 1.098 1.104 1.11   1.116 1.122 1.128 1.134 1.14   1.146
1.152 1.158 1.164 1.17   1.176 1.182 1.188 1.194]
Valores de las constantes h: 0.006 y k:0.0004, k/h: 0.06666666666666667
```

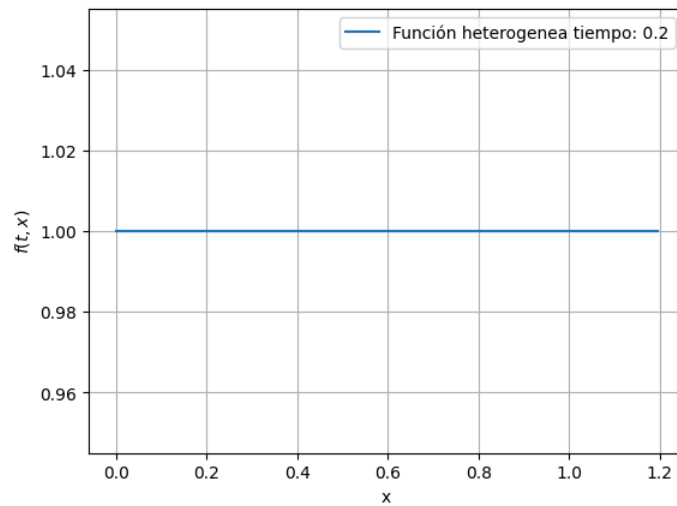
```
1 # Función f(t,x)
2 def funcion_heterogenea(
3     x,
4     t,
5 ):
6     """
7     Función heterogenea de la Ecuación Diferencial Parcial.
8     t : float o np.ndarray
9     x : float o np.ndarray
10    """
11    t = np.array(t)
12    x = np.array(x)
13
14    return np.piecewise(x, [x < 0, (x >= 0)], [0, 1])
15
16
17 f = funcion_heterogenea(variable_espacial, 0.2)
18 plt.plot(variable_espacial, f, label=f"Función heterogenea tiempo: {0.2}")
19 plt.xlabel("x")
20 plt.ylabel("$f(t, x)$")
```

```

21 plt.grid(True)
22 plt.legend()

```

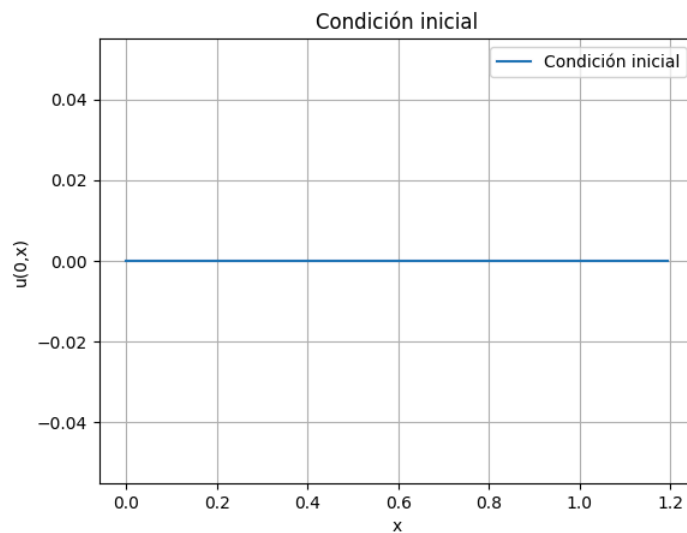
<matplotlib.legend.Legend at 0x19c3c58eab0>



```

1 # --- Condición inicial u(0,x) = 0 ---
2 def condicion_inicial(
3     x,
4     t,
5 ):
6     return np.zeros(len(x))
7
8
9 cond_iniciales = condicion_inicial(variable_espacial, 0)
10 plt.plot(variable_espacial, cond_iniciales, label="Condición inicial")
11 plt.xlabel("x")
12 plt.ylabel("u(0,x)")
13 plt.grid(True)
14 plt.title("Condición inicial")
15 plt.legend()
16 plt.show()

```



✓ Solución exacta

```

1 def solucion_exacta(
2     x, # Variable espacial
3     t, # Variable temporal
4 ):
5     """
6     Solución excata al PVI dado anterior mente.
7     t : float o np.ndarray
8     x : float o np.ndarray
9     """
10    t = np.array(t)
11    x = np.array(x)

```

```

12
13     return np.piecewise(
14         x,
15         [x <= 0, (x >= 0) & (x - 0.5 * t <= 0), (x >= 0) & (x - 0.5 * t >= 0)],
16         [0, lambda x: 2 * x, lambda x: t],
17     )

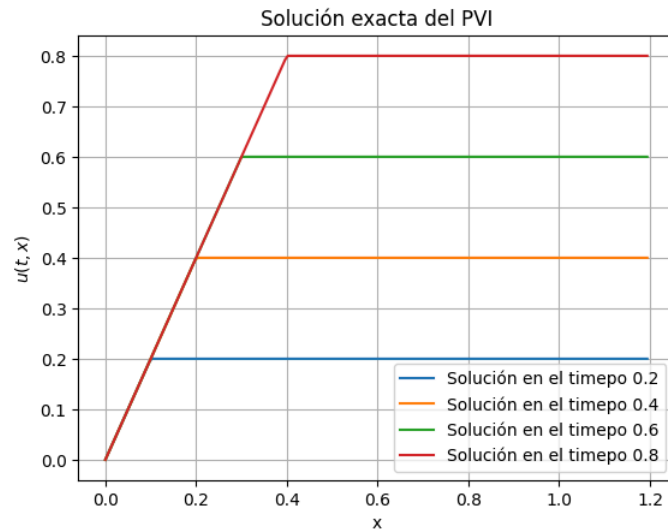
```

```

1 # Graficar la solución exacta del PVI
2 for t in t_comparación:
3     u = solucion_exacta(variable_espacial, t)
4     plt.plot(variable_espacial, u, label=f"Solución en el tiempoo {t}")
5
6 plt.title("Solución exacta del PVI")
7 plt.xlabel("x")
8 plt.ylabel("$u(t, x)$")
9 plt.grid(True)
10 plt.legend()

```

<matplotlib.legend.Legend at 0x19c3c68a630>



✓ b. Adelante en tiempo - cénrado en espacio

El esquema dado para adelante en tiempo - centrado en espacio está dado por:

$$\frac{V_m^{n+1} - V_m^n}{k} + a \frac{V_{m+1}^n - V_{m-1}^n}{2h} = f_m^n$$

Despejando para el tiempo t_{n+1} :

$$V_m^{n+1} = V_m^n + \frac{ak}{2h} (V_{m-1}^n - V_{m+1}^n) + k f_m^n \text{ con: } m = 1, \dots, M-1 \text{ y } n = 0, \dots, N-1$$

Se puede evidenciar que este esquema es un esquema explícita y de a un paso

Resolviendo el sistema lineal de ecuaciones, nos damos cuenta que se puede resolver como una sistema matricial:

$$V^{n+1} = AV^n + \tilde{F}$$

Lo que nos da la matriz A como:

$$A = \begin{bmatrix} 1 & -\frac{ak}{2h} & 0 & 0 & 0 & 0 \\ \frac{ak}{2h} & 1 & -\frac{ak}{2h} & 0 & 0 & \vdots \\ 0 & \frac{ak}{2h} & 1 & -\frac{ak}{2h} & 0 & \vdots \\ 0 & 0 & \frac{ak}{2h} & \ddots & \ddots & 0 \\ \vdots & \vdots & 0 & \ddots & \ddots & -\frac{ak}{2h} \\ 0 & 0 & 0 & 0 & \frac{ak}{2h} & 1 \end{bmatrix}$$

$$\tilde{F} = \begin{bmatrix} k f_m^n + \frac{ak}{2h} u(t, 0) \\ k f_m^n \\ k f_m^n \\ k f_m^n \\ \vdots \\ k f_m^n - \frac{ak}{2h} u(t, 1.2) \end{bmatrix}$$

Handwritten notes: f_m^n (circled), $k f_m^2$, $k f_m^3$, $k f_m^4$, $k f_m^n$, $u(t, n, +2)$, $k f_{m-1}^n$, $-\frac{ak}{2h} V_m^n$.

Para la construcción de la matriz A solo es necesario el almacenamiento de la diagonal y las diagonales superior e inferior, ya que el resto de los valores son 0, los cuales no brindan información.

Estabilidad

$$|a+b|^2 \leq 2(|a|^2 + |b|^2)$$

Analizando la estabilidad del sistema, tenemos que:

$$\begin{aligned} V_m^{n+1} &= V_m^n + \frac{a\lambda}{2}(V_{m-1}^n - V_{m+1}^n) \\ \Rightarrow |V_m^{n+1}|^2 &= |V_m^n + \frac{a\lambda}{2}(V_{m-1}^n - V_{m+1}^n)|^2 \\ \Rightarrow |V_m^{n+1}|^2 &\leq |V_m^n|^2 + \frac{|a\lambda|^2}{2}(|V_{m-1}^n - V_{m+1}^n|)^2 \\ &= |V_m^n|^2 + \frac{|a\lambda|^2}{2}(|V_{m-1}^n|^2 - 2|V_{m-1}^n||V_{m+1}^n| + |V_{m+1}^n|^2) \\ &= |V_m^n|^2 + \frac{|a\lambda|^2}{2}(|V_{m-1}^n|^2 - 2|V_{m+1}^n||V_{m-1}^n| + |V_{m+1}^n|^2) \\ &= |V_m^n|^2 + \frac{|a\lambda|^2}{2}|V_{m-1}^n|^2 - |a\lambda|^2|V_{m+1}^n||V_{m-1}^n| + \frac{|a\lambda|^2}{2}|V_{m+1}^n|^2 \\ &\leq |V_m^n|^2 + \frac{|a\lambda|^2}{2}|V_{m-1}^n|^2 - |a\lambda|^2(|V_{m+1}^n|^2 + |V_{m-1}^n|^2) + \frac{|a\lambda|^2}{2}|V_{m+1}^n|^2 \\ &= |V_m^n|^2 - \frac{|a\lambda|^2}{2}|V_{m-1}^n|^2 - \frac{|a\lambda|^2}{2}|V_{m+1}^n|^2 \end{aligned}$$

Handwritten notes: *no debe quedar un menos*, $|a-b|^2 \leq (|a| + |b|)^2 \leq 2(|a|^2 + |b|^2)$.

Con lo que

$$\begin{aligned} \sum_{m=-\infty}^{\infty} |V_m^{n+1}|^2 &\leq \sum_{m=-\infty}^{\infty} |V_m^n|^2 - \frac{|a\lambda|^2}{2}(\sum_{m=-\infty}^{\infty} |V_{m-1}^n|^2 + \sum_{m=-\infty}^{\infty} |V_{m+1}^n|^2) \\ &= \sum_{m=-\infty}^{\infty} |V_m^n|^2 - \frac{|a\lambda|^2}{2}(2 \sum_{m=-\infty}^{\infty} |V_m^n|^2) \\ \Rightarrow \sum_{m=-\infty}^{\infty} |V_m^{n+1}|^2 &\leq (1 - |a\lambda|^2) \sum_{m=-\infty}^{\infty} |V_m^n|^2 \\ \Rightarrow \sum_{m=-\infty}^{\infty} |V_m^{n+1}|^2 &\leq (1 - |a\lambda|^2)^{n+1} \sum_{m=-\infty}^{\infty} |V_m^0|^2 \end{aligned}$$

Se debe de cumplir que $1 - (a\lambda)^2 \leq 1$ cuando $n \rightarrow \infty$ para que no "explote", por lo que se debe de cumplir que:

$$1 - (a\lambda)^2 \leq 1 \Rightarrow (a\lambda)^2 \geq 0$$

Handwritten note: *Acá se llega un esquema inmutable*.

Como $(a\lambda)^2$ siempre va a ser un número positivo, el esquema siempre va a ser estable.

1 cons_DC = (alpha * k) / (2 * h)

```
1 class EsquemaAdelanteCentrado(Esquema):
2     def __init__(
3         self,
4         valor_M,
5         valor_N,
6         valor_k,
7         constante_matriz,
8     ):
9         self.valor_M = valor_M
10        self.valor_N = valor_N
11        self.valor_k = valor_k
12        self.constante_matriz = constante_matriz
13
14    def calcular_esquema(self):
15        diag = np.ones(self.valor_M)
16        diag_inf = np.zeros(self.valor_M - 1) + self.constante_matriz
17        diag_sup = np.zeros(self.valor_M - 1) - self.constante_matriz
18
19        # Factorización de la matriz
20        diag_L, diag_inf_L, diag_sup_U = matrix_factorization_nxn(
21            diagonal=diag, diagonal_superior=diag_sup, diagonal_inferior=diag_inf
22        )
23
24        V = np.zeros((self.valor_N + 1, self.valor_M))
25
26        # Condiciones iniciales
27        cond_iniciales = condicion_inicial(x=variable_espacial, t=variable_temporal[0])
```

¿qué no se resuelve sistema?

```

28     V[0, :] = cond_iniciales
29
30     for i in range(1, N + 1):
31         # Construcción del vector F
32         F = (
33             funcion_heterogenea(x=variable_espacial, t=variable_temporal[i])
34             * self.valor_k
35         )
36
37         # Definición de las condiciones de frontera (inferior y superior)
38         condicion_frontera_inf = 0
39         condicion_frontera_sup = variable_temporal[i]
40
41         # Ciclo for para encontrar el valor de V en el tiempo i y en todos los puntos del espacio
42         for j in range(1, self.valor_M):
43             # Cálculo del valor de V teniendo en cuenta la condición de frontera inferior
44             if j == 1:
45                 V[i, j] = (
46                     V[i - 1, j]
47                     + self.constante_matriz
48                     * (condicion_frontera_inf - V[i - 1, j + 1])
49                     + F[j]
50                 )
51             # Cálculo del valor de V teniendo en cuenta la condición de frontera superior
52             elif j == self.valor_M - 1:
53                 V[i, j] = (
54                     V[i - 1, j]
55                     + self.constante_matriz
56                     * (V[i - 1, j - 1] - condicion_frontera_sup)
57                     + F[j]
58                 )
59             # Cálculo de V en los puntos intermedios
60             else:
61                 V[i, j] = (
62                     V[i - 1, j]
63                     + self.constante_matriz * (V[i - 1, j - 1] - V[i - 1, j + 1])
64                     + F[j]
65                 )
66
67     return V

```

```

1 esquema_DC = EsquemaAdelanteCentrado(
2     valor_M=M,
3     valor_N=N,
4     valor_k=k,
5     constante_matriz=cons_DC,
6 )
7
8 name_esquema = "Esquema adelante tiempo - centrado espacio"
9
10 animar_esquema(
11     N=N,
12     esquema=esquema_DC,
13     condicion_inicial=cond_iniciales,
14     filename=name_esquema,
15 )

```

✓ Animación guardada en Esquema adelante tiempo - centrado espacio
<matplotlib.animation.FuncAnimation at 0x19c3d7c0350>

▼ Comparación con la solución exacta

```

1 solucion_DF = esquema_DC.calcular_esquema()
2
3 indices = [np.where(variable_temporal == t)[0][0] for t in t_comparación]
4
5
6 for pos, t in enumerate(t_comparación):
7     u = solucion_exacta(variable_espacial, t)
8     plt.plot(variable_espacial, u, label=f"Solución en el tiempoo {t}")
9     plt.plot(
10         variable_espacial,
11         solucion_DF[indices[pos], :],
12         label=f"Solución numérica en el tiempoo {t}",
13         linestyle="dashed",
14     )
15     rmsd = RMSD(u, solucion_DF[indices[pos], :])
16     print(f"RMSD del {name_esquema} en t={t}: {round(rmsd,5)}")
17
18 plt.title(
19     "Comparación solución exacta y numérica: Esquema adelante tiempo - centrado espacio"
20 )
21 plt.xlabel("x")

```

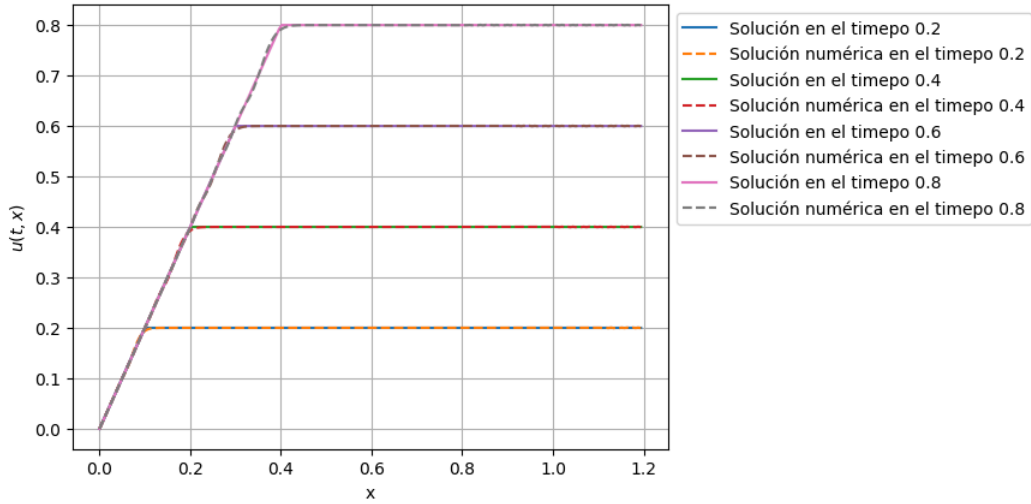
```

22 plt.ylabel("$u(t, x)$")
23 plt.grid(True)
24 plt.legend(loc="upper left", bbox_to_anchor=(1, 1))
25 plt.show()

```

RMSD del Esquema adelante tiempo - centrado espacio en $t=0.2$: 0.00078
 RMSD del Esquema adelante tiempo - centrado espacio en $t=0.4$: 0.0012
 RMSD del Esquema adelante tiempo - centrado espacio en $t=0.6$: 0.0016
 RMSD del Esquema adelante tiempo - centrado espacio en $t=0.8$: 0.00189

Comparación solución exacta y numérica: Esquema adelante tiempo - centrado espacio



Tener valores de
 $h=k=0.01$
 ¿qué obtenemos?

Se puede concluir que la solución numérica del esquema adelante en tiempo y centrado en el espacio entrega una buena aproximación al Problema de Valores Iniciales (PVI) propuesto. Esto se evidencia en la comparación visual realizada en los intervalos de tiempo y por el valor numérico obtenido por el RMSD el cual es muy cercano a cero, indicando que la aproximación realizada por el esquema toma valores muy cercanos a la solución exacta.

✓ e. Atrás en tiempo - atrás en espacio

$$\frac{V_m^n - V_m^{n-1}}{k} + a \frac{V_m^n - V_{m-1}^n}{h} = f_m^n$$

Despejando el tiempo nuevo t_n en terminos del viejo tiempo t_{n-1} , se obtiene:

$$(1 + \frac{ka}{h})V_m^n - \frac{ka}{h}V_{m-1}^n = kf_m^n + V_m^{n-1} \text{ con: } m = 1, \dots, M-1 \text{ y } n = 1, \dots, N$$

Resolviendo el sistema lineal de ecuaciones, nos damos cuenta que se puede resolver como una sistema matricial:

$$AV^n = V^{n-1} + \tilde{F}$$

Lo que nos da la matriz A como:

$$A = \begin{bmatrix} 1 + \frac{ak}{h} & 0 & 0 & 0 & 0 & 0 \\ -\frac{ak}{h} & 1 + \frac{ak}{h} & 0 & 0 & 0 & \vdots \\ 0 & -\frac{ak}{h} & 1 + \frac{ak}{h} & 0 & 0 & \vdots \\ 0 & 0 & -\frac{ak}{h} & 1 + \frac{ak}{h} & 0 & \vdots \\ \vdots & \vdots & 0 & \ddots & \ddots & 0 \\ 0 & 0 & 0 & 0 & -\frac{ak}{h} & 1 + \frac{ak}{h} \end{bmatrix}$$

$u(t, 0)$

$$\tilde{F} = \begin{bmatrix} kf_m^n + \frac{ak}{h}u(t, 0) \\ kf_m^n \\ kf_m^n \\ \vdots \\ kf_m^n \end{bmatrix}$$

De nuevo f_m^n
 debe cambiarse
 en cada fila

✓ Estabilidad

Analizando la estabilidad del esquema nos damos cuenta que $a\lambda \geq 0$

Demostración:

$$\begin{aligned}
 V_m^n - V_m^{n-1} + a\lambda(V_m^n - V_{m-1}^n) &= 0 \\
 \implies (1 + a\lambda)V_m^n &= V_m^{n-1} + a\lambda V_{m-1}^n \\
 \implies (1 + a\lambda)^2 |V_m^n|^2 &= |V_m^{n-1} + a\lambda V_{m-1}^n|^2 \\
 \implies (1 + a\lambda)^2 |V_m^n|^2 &\leq |V_m^{n-1}|^2 + (a\lambda)^2 |V_{m-1}^n|^2 \\
 \implies (1 + a\lambda)^2 \sum_{m=-\infty}^{\infty} |V_m^n|^2 &\leq \sum_{m=-\infty}^{\infty} |V_m^{n-1}|^2 + (a\lambda)^2 \sum_{m=-\infty}^{\infty} |V_{m-1}^n|^2 \\
 \implies (1 + a\lambda)^2 \sum_{m=-\infty}^{\infty} |V_m^n|^2 &\leq \sum_{m=-\infty}^{\infty} |V_m^{n-1}|^2 + (a\lambda)^2 \sum_{m=-\infty}^{\infty} |V_m^n|^2 \\
 \implies (1 + a\lambda)^2 \sum_{m=-\infty}^{\infty} |V_m^n|^2 - (a\lambda)^2 \sum_{m=-\infty}^{\infty} |V_m^n|^2 &\leq \sum_{m=-\infty}^{\infty} |V_m^{n-1}|^2 \\
 \implies [1 + 2a\lambda + (a\lambda)^2 - (a\lambda)^2] \sum_{m=-\infty}^{\infty} |V_m^n|^2 &\leq \sum_{m=-\infty}^{\infty} |V_m^{n-1}|^2 \\
 \implies \sum_{m=-\infty}^{\infty} |V_m^n|^2 &\leq \frac{1}{[1 + 2a\lambda]} \sum_{m=-\infty}^{\infty} |V_m^{n-1}|^2 \\
 \implies \sum_{m=-\infty}^{\infty} |V_m^n|^2 &\leq \frac{1}{[1 + 2a\lambda]^{n+1}} \sum_{m=-\infty}^{\infty} |V_m^0|^2
 \end{aligned}$$

En este se demuestra que para $a\lambda \geq 0$ es incondicionalmente estable

Se debe de cumplir que $\frac{1}{[1 + 2a\lambda]^{n+1}} \leq 1$ cuando $n \rightarrow \infty$ para que no "explote", por lo que se debe de cumplir que:

$$1 + 2a\lambda \geq 1 \implies 2a\lambda \geq 0 \implies a\lambda \geq 0$$

```

1 # Constante atras-atras
2 cons_AA = (alpha * k) / h
3

```

```

1 class EsquemaAtrasAtras(Esquema):
2     def __init__(
3         self,
4         valor_M,
5         valor_N,
6         valor_k,
7         constante_matriz,
8     ):
9         self.valor_M = valor_M
10        self.valor_N = valor_N
11        self.valor_k = valor_k
12        self.constante_matriz = constante_matriz
13
14    def calcular_esquema(self):
15        diag = np.ones(self.valor_M) + self.constante_matriz
16        diag_inf = np.zeros(self.valor_M - 1) - self.constante_matriz
17
18        V = np.zeros((self.valor_N + 1, self.valor_M))
19
20        # Condiciones iniciales
21        cond_iniciales = condicion_inicial(x=variable_espacial, t=variable_temporal[0])
22        # print(f"condicion inicial: {cond_iniciales}")
23        V[0, :] = cond_iniciales
24
25        for t in range(1, self.valor_N + 1):
26            # Construcción del vector F
27            F = (
28                funcion_heterogenea(x=variable_espacial, t=variable_temporal[t])
29                * self.valor_k
30            )
31            # print(F.shape)
32            # Definición de la condición de frontera inferior
33            condicion_frontera_inf = 0
34            # Adición del valor en la frontera
35            F[0] = F[0] + self.constante_matriz * condicion_frontera_inf
36            # Valores anteriores
37            V_anterior = V[t - 1, :] + F
38            # print(f" Valor anterior: {V_anterior}")
39            V[t, :] = sustitucion_progresiva(
40                l_diagonal=diag,
41                l_diagonal_inferior=diag_inf,
42                termino_independiente=V_anterior,
43            )
44            # print(f"Valor posterior: {V[t, :]}")
45
46        return V

```

```

1 esquema_AA = EsquemaAtrasAtras(
2     valor_M=M,
3     valor_N=N,
4     valor_k=k,
5     constante_matriz=cons_AA,

```

```

6 )
7
8 name_esquema = "Esquema atras tiempo - atras espacio"
9
10 animar_esquema(
11     N=N,
12     esquema=esquema_AA,
13     condicion_inicial=cond_iniciales,
14     filename=name_esquema,
15 )

```

✓ Animación guardada en Esquema atras tiempo - atras espacio
 <matplotlib.animation.FuncAnimation at 0x19c3d847bc0>

✓ Comparación con la solución exacta

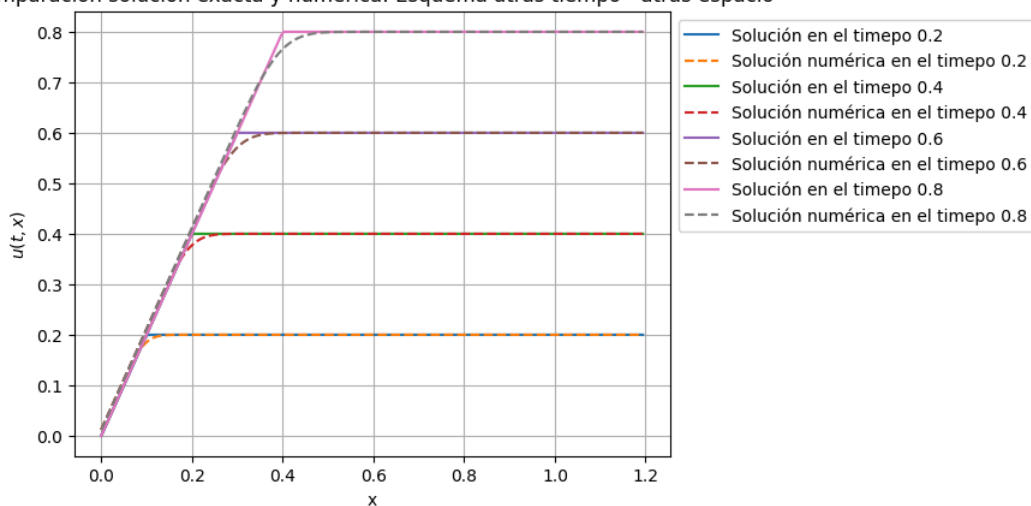
```

1 solucion_DF = esquema_AA.calcular_esquema()
2
3 indices = [np.where(variable_temporal == t)[0][0] for t in t_comparación]
4
5
6 for pos, t in enumerate(t_comparación):
7     u = solucion_exacta(variable_espacial, t)
8     plt.plot(variable_espacial, u, label=f"Solución en el tiempoo {t}")
9     plt.plot(
10         variable_espacial,
11         solucion_DF[indices[pos], :],
12         label=f"Solución numérica en el tiempoo {t}",
13         linestyle="dashed",
14     )
15     rmsd = RMSD(u, solucion_DF[indices[pos], :])
16     print(f"RMSD del {name_esquema} en t={t}: {round(rmsd,5)}")
17
18 plt.title(
19     "Comparación solución exacta y numérica: Esquema atras tiempo - atras espacio"
20 )
21 plt.xlabel("x")
22 plt.ylabel("$u(t, x)$")
23 plt.grid(True)
24 plt.legend(loc="upper left", bbox_to_anchor=(1, 1))
25 plt.show()

```

RMSD del Esquema atras tiempo - atras espacio en t=0.2: 0.00326
 RMSD del Esquema atras tiempo - atras espacio en t=0.4: 0.00514
 RMSD del Esquema atras tiempo - atras espacio en t=0.6: 0.00676
 RMSD del Esquema atras tiempo - atras espacio en t=0.8: 0.00817

Comparación solución exacta y numérica: Esquema atras tiempo - atras espacio



De los resultados obtenidos de la comparación entre la solución exacta y la solución numerica del esquema atrás en tiempo y atrás en el espacio, se evidencia que el esquema propuesto entrega valores muy cercanos a la solución exacta del PVI, aunque presenta pequeñas diferencias en los puntos de unión de la solución exacta. Esta pequeña discrepancia se puede mejorar aumentando la cantidad de puntos utilizados en la discretización de la variable espacial.

✓ f. Atrás en tiempo - centrado en espacio

$$\frac{V_m^n - V_m^{n-1}}{k} + a \frac{V_{m+1}^n - V_{m-1}^n}{2h} = f_m^n$$

Despejando el tiempo nuevo t_n en terminos del viejo tiempo t_{n-1} , se obtiene:

$$V_m^n + \frac{ka}{2h} V_{m+1}^n - \frac{ka}{2h} V_{m-1}^n = k f_m^n + V_m^{n-1} \text{ con: } m = 1, \dots, M-1 \text{ y } n = 1, \dots, N$$

Resolviendo el sistema lineal de ecuaciones, nos damos cuenta que se puede resolver como una sistema matricial:

$$AV^n = V^{n-1} + \tilde{F}$$

Lo que nos da la matriz A como:

$$A = \begin{bmatrix} 1 & \frac{ak}{2h} & 0 & 0 & 0 & 0 \\ -\frac{ak}{2h} & 1 & \frac{ak}{2h} & 0 & 0 & \vdots \\ 0 & -\frac{ak}{2h} & 1 & \frac{ak}{2h} & 0 & \vdots \\ 0 & 0 & -\frac{ak}{2h} & \ddots & \ddots & 0 \\ \vdots & \vdots & 0 & \ddots & \ddots & \frac{ak}{2h} \\ 0 & 0 & 0 & 0 & -\frac{ak}{2h} & 1 \end{bmatrix}$$

(f_m^n) $\tilde{F} = \begin{bmatrix} kf_m^n + \frac{ak}{2h} u(t, 0) \\ kf_m^n \\ kf_m^n \\ kf_m^n \\ \vdots \\ kf_m^n - \frac{ak}{2h} u(t, 1.2) \end{bmatrix}$ $u(t_n, 0) = V_0^n$
 $u(t_n, 1.2) = V_M^n$

✓ Estabilidad

Realizando el análisis de estabilidad con Von-Neumann nos damos cuenta que este esquema es incondicionalmente estable

Demostración:

Realizando el cambio de V_m^n por $g^n e^{im\theta}$:

$$\Rightarrow g^n e^{im\theta} - g^{n-1} e^{im\theta} + \frac{a\lambda}{2} (g^n e^{i(m+1)\theta} - g^n e^{i(m-1)\theta}) = 0$$

$$\Rightarrow g^n e^{im\theta} [1 - g^{-1} + \frac{a\lambda}{2} (e^{i\theta} - e^{-i\theta})] = 0$$

$$\Rightarrow g^n e^{im\theta} [1 - g^{-1} + \frac{a\lambda}{2} (2i \sin \theta)] = 0$$

$$\Rightarrow g^n e^{im\theta} [1 - g^{-1} + a\lambda i \sin \theta] = 0$$

Despejando g :

$$\Rightarrow 1 - g^{-1} + a\lambda i \sin \theta = 0$$

$$\Rightarrow g = \frac{1}{1 + a\lambda i \sin \theta}$$

Sacando $|g|^2$ obtenemos que:

$$\Rightarrow |g|^2 = \frac{1}{1 + (a\lambda)^2 \sin^2 \theta} \quad \checkmark$$

Se debe de cumplir que $|g|^2 \leq 1$: $\checkmark$

$$\Rightarrow \frac{1}{1 + (a\lambda)^2 \sin^2 \theta} \leq 1$$

$$\Rightarrow 1 + (a\lambda)^2 \sin^2 \theta \geq 1$$

y para ello se debe cumplir $1 + (a\lambda \sin \theta)^2 \geq 1$
 y esto siempre se cumple

no es correcta esta escritura

Como $0 \leq \sin^2 \theta \leq 1$ y $(a\lambda)^2 > 0$, el esquema va a ser incondicionalmente estable.

```
1 # Definición de la constante
2 cons_AC = (alpha * k) / (2 * h)
```

```
1 class EsquemaAtrasCentrado(Esquema):
2     def __init__(
3         self,
4         valor_M,
5         valor_N,
6         valor_k,
7         constante_matriz,
8     ):
9         self.valor_M = valor_M
10        self.valor_N = valor_N
```

```

11     self.valor_k = valor_k
12     self.constante_matriz = constante_matriz
13
14     def calcular_esquema(self):
15         diag = np.ones(self.valor_M)
16         diag_inf = np.zeros(self.valor_M - 1) - self.constante_matriz
17         diag_sup = np.zeros(self.valor_M - 1) + self.constante_matriz
18
19         # Factorización de la matriz
20         diag_L, diag_inf_L, diag_sup_U = matrix_factorization_nxn(
21             diagonal=diag, diagonal_superior=diag_sup, diagonal_inferior=diag_inf
22         )
23
24         V = np.zeros((self.valor_N + 1, self.valor_M))
25
26         # Condiciones iniciales
27         cond_iniciales = condicion_inicial(x=variable_espacial, t=variable_temporal[0])
28         V[0, :] = cond_iniciales
29         for t in range(1, N + 1):
30             # Construcción del vector F
31             F = (
32                 funcion_heterogenea(x=variable_espacial, t=variable_temporal[t])
33                 * self.valor_k
34             )
35
36             # Definición de la condición de frontera inferior
37             condicion_frontera_inf = 0
38             # Definición de la condición de frontera superior
39             condicion_frontera_sup = variable_temporal[t]
40
41             # Adición del valor en la frontera
42             F[0] = F[0] + self.constante_matriz * condicion_frontera_inf
43             F[-1] = F[-1] - self.constante_matriz * condicion_frontera_sup
44             # print(F)
45             # Valores anteriores
46             V_anterior = V[t - 1, :] + F
47
48             # Aplicación de la sustitución regresiva
49             z = sustitucion_regresiva(
50                 u_diagonal_superior=diag_sup_U, termino_independiente=V_anterior
51             )
52
53             # Aplicación de la sustitución progresiva
54             V[t, :] = (
55                 sustitucion_progresiva(
56                     l_diagonal=diag_L,
57                     l_diagonal_inferior=diag_inf_L,
58                     termino_independiente=z,
59                 )
60                 # + F
61             )
62
63     return V

```

```

1 esquema_AC = EsquemaAtrasCentrado(
2     valor_M=M,
3     valor_N=N,
4     valor_k=k,
5     constante_matriz=cons_AC,
6 )
7
8 name_esquema = "Esquema atras tiempo - centrado espacio"
9
10 animar_esquema(
11     N=N,
12     esquema=esquema_AC,
13     condicion_inicial=cond_iniciales,
14     filename=name_esquema,
15 )

```

 Animación guardada en Esquema atras tiempo - centrado espacio
 <matplotlib.animation.FuncAnimation at 0x19c001c1b20>

Comparación solución exacta

```

1 solucion_DF = esquema_AC.calcular_esquema()
2
3 indices = [np.where(variable_temporal == t)[0][0] for t in t_comparación]
4
5 for pos, t in enumerate(t_comparación):
6     u = solucion_exacta(variable_espacial, t)
7     plt.plot(variable_espacial, u, label=f"Solución en el tiempo {t}")
8     plt.plot(

```

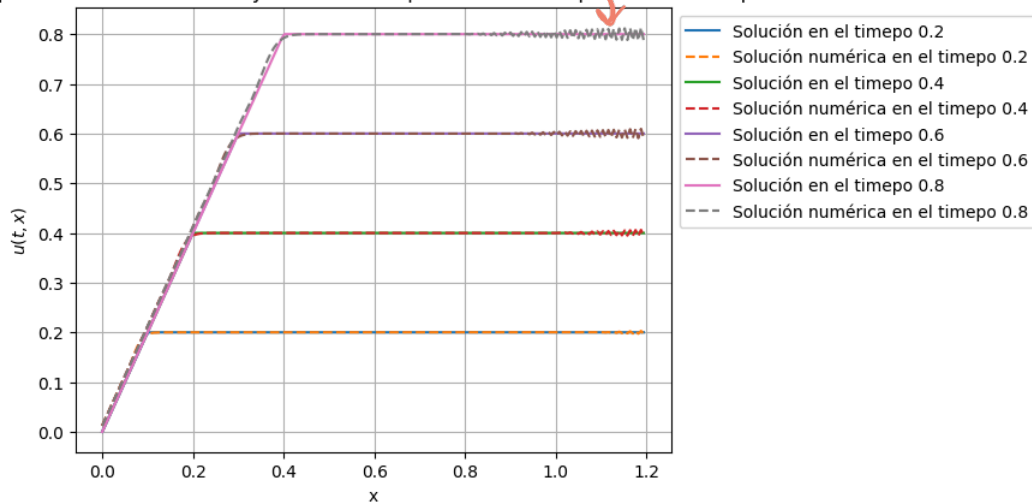
```

9     variable_espacial,
10    solucion_DF[indices[pos], :],
11    label=f"Solución numérica en el tiempo {t}",
12    linestyle="dashed",
13 )
14 rmsd = RMSD(u, solucion_DF[indices[pos], :])
15 print(f"RMSD del {name_esquema} en t={t}: {round(rmsd,5)}")
16
17 plt.title(
18     "Comparación solución exacta y numérica: Esquema atras tiempo - centrada espacio"
19 )
20 plt.xlabel("x")
21 plt.ylabel("$u(t, x)$")
22 plt.grid(True)
23 plt.legend(loc="upper left", bbox_to_anchor=(1, 1))
24 plt.show()

```

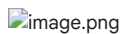
RMSD del Esquema atras tiempo - centrado espacio en t=0.2: 0.00365
 RMSD del Esquema atras tiempo - centrado espacio en t=0.4: 0.00534
 RMSD del Esquema atras tiempo - centrado espacio en t=0.6: 0.00687
 RMSD del Esquema atras tiempo - centrado espacio en t=0.8: 0.00843

Comparación solución exacta y numérica: Esquema atras tiempo - centrada espacio



De la comparación mostrada y de los valores obtenidos del RMSD, se puede concluir que el esquema atrás en tiempo y centrado en espacio entrega una buena aproximación para el PVI propuesto, pero este esquema presenta un pequeño error en la condición de frontera superior, el cual se va a ir propagando durante el tiempo y se ira incrementando mediante el avance de cada iteración temporal. Lo cual puede generar que el esquema no realice buenas aproximaciones en un intervalo de tiempo más prolongado. Para mirar cuales son las causas y posibles soluciones a esta propagación de errores se debe de realizar un estudio más exhaustivo en el algoritmo utilizado para la programación de este esquema. Este pequeño error también se puede deber a la discretización realizada, lo cual amerita un estudio para encontrar la relación idónea entre la discretización temporal y espacial.

✓ Ejercicio 2. ¹⁸/₂₀



Tenemos que el esquema es consistente si para cualquier función $\varphi(t, x)$ "suave" se cumple que:

$$P\varphi - P_{kh}\varphi \longrightarrow 0 \text{ cuando } h, k \longrightarrow 0$$

Esto es:

$$(P\varphi)_m^n - P_{kh}\varphi_m^n \longrightarrow 0 \text{ cuando } h, k \longrightarrow 0$$

Tenemos que:

$$(\varphi_t)_m^n + c(\varphi_{tx})_m^n + a(\varphi_x)_m^n - \left[\frac{\varphi_m^{n+1} - \varphi_m^n}{k} + c \frac{\varphi_{m+1}^{n+1} - \varphi_{m-1}^{n+1} - \varphi_{m+1}^n + \varphi_{m-1}^n}{2kh} + a \frac{\varphi_{m+1}^n - \varphi_{m-1}^n}{2h} \right]$$

Para la derivada temporal:

$$\begin{aligned}
 \varphi_m^{n+1} &= \varphi_m^n + k(\varphi_t)_m^n + \frac{k^2}{2}(\varphi_{tt})_m^n + O(k^3) \\
 \implies \varphi_m^{n+1} - \varphi_m^n &= k(\varphi_t)_m^n + \frac{k^2}{2}(\varphi_{tt})_m^n + O(k^3) \\
 \implies \frac{\varphi_m^{n+1} - \varphi_m^n}{k} &= (\varphi_t)_m^n + \frac{k}{2}(\varphi_{tt})_m^n + O(k^2)
 \end{aligned}$$

$$\therefore (\varphi_t)_m^n - \frac{\varphi_{m+1}^{n+1} - \varphi_m^n}{k} = \frac{k}{2} (\varphi_{tt})_m^n + O(k^2)$$

Para la derivada espacial:

$$\varphi_{m\pm 1}^n = \varphi_m^n \pm h(\varphi_x)_m^n + \frac{h^2}{2}(\varphi_{xx})_m^n \pm \frac{h^3}{3!}(\varphi_{xxx})_m^n + \frac{h^4}{4!}(\varphi_{xxxx})_m^n + O(h^5)$$

Restando $\varphi_{m+1}^n - \varphi_{m-1}^n$ se obtiene:

$$\begin{aligned} \varphi_{m+1}^n - \varphi_{m-1}^n &= 2h(\varphi_x)_m^n + 2\frac{h^3}{3!}(\varphi_{xxx})_m^n + O(h^5) \\ \implies \frac{\varphi_{m+1}^n - \varphi_{m-1}^n}{2h} &= (\varphi_x)_m^n + \frac{h^2}{3!}(\varphi_{xxx})_m^n + O(h^4) \\ \therefore (\varphi_x)_m^n - \frac{\varphi_{m+1}^{n+1} - \varphi_{m-1}^{n+1}}{2h} &= \frac{h^2}{3!}(\varphi_{xxx})_m^n + O(h^4) \end{aligned}$$

Para la derivada mixta, vamos a ver que sucede con φ_{m+1}^{n+1} y φ_{m-1}^{n+1} :

- $\varphi_{m+1}^{n+1} = \varphi_{m+1}^n + k(\varphi_t)_{m+1}^n + \frac{k^2}{2}(\varphi_{tt})_{m+1}^n + \frac{k^3}{3!}(\varphi_{ttt})_{m+1}^n + O(k^4)$
- $\varphi_{m-1}^{n+1} = \varphi_{m-1}^n + k(\varphi_t)_{m-1}^n + \frac{k^2}{2}(\varphi_{tt})_{m-1}^n + \frac{k^3}{3!}(\varphi_{ttt})_{m-1}^n + O(k^4)$

Restando $\varphi_{m+1}^{n+1} - \varphi_{m-1}^{n+1}$ se obtiene:

$$\begin{aligned} \varphi_{m+1}^{n+1} - \varphi_{m-1}^{n+1} &= \varphi_{m+1}^n - \varphi_{m-1}^n + k[(\varphi_t)_{m+1}^n - (\varphi_t)_{m-1}^n] + \frac{k^2}{2}[(\varphi_{tt})_{m+1}^n - (\varphi_{tt})_{m-1}^n] + \frac{k^3}{3!}[(\varphi_{ttt})_{m+1}^n - (\varphi_{ttt})_{m-1}^n] + O(k^4) \\ \implies \varphi_{m+1}^{n+1} - \varphi_{m-1}^{n+1} - \varphi_{m+1}^n + \varphi_{m-1}^n &= k[(\varphi_t)_{m+1}^n - (\varphi_t)_{m-1}^n] + \frac{k^2}{2}[(\varphi_{tt})_{m+1}^n - (\varphi_{tt})_{m-1}^n] + \frac{k^3}{3!}[(\varphi_{ttt})_{m+1}^n - (\varphi_{ttt})_{m-1}^n] \\ &+ O(k^4) \end{aligned}$$

Al observar las derivadas $(\varphi_t)_{m\pm 1}^n$:

- $(\varphi_t)_{m\pm 1}^n = (\varphi_t)_m^n \pm h((\varphi_{tx}))_m^n + \frac{h^2}{2}(\varphi_{txx})_m^n \pm \frac{h^3}{3!}(\varphi_{txxx})_m^n + O(h^4)$

Restando $(\varphi_t)_{m+1}^n - (\varphi_t)_{m-1}^n$:

$$\begin{aligned} (\varphi_t)_{m+1}^n - (\varphi_t)_{m-1}^n &= 2h(\varphi_{tx})_m^n + 2\frac{h^3}{3!}(\varphi_{txxx})_m^n + O(h^5) \\ \implies (\varphi_t)_{m+1}^n - (\varphi_t)_{m-1}^n - 2h(\varphi_{tx})_m^n &= \frac{h^2}{3!}(\varphi_{txxx})_m^n + O(h^4) \end{aligned}$$

Sustituyendo $(\varphi_t)_{m+1}^n - (\varphi_t)_{m-1}^n$ en $\varphi_{m+1}^{n+1} - \varphi_{m-1}^{n+1} - \varphi_{m+1}^n + \varphi_{m-1}^n$, se obtiene:

$$\begin{aligned} \varphi_{m+1}^{n+1} - \varphi_{m-1}^{n+1} - \varphi_{m+1}^n + \varphi_{m-1}^n &= 2hk[(\varphi_{tx})_m^n + \frac{h^2}{3!}(\varphi_{txxx})_m^n + O(h^4)] + \frac{k^2}{2}[(\varphi_{tt})_{m+1}^n - (\varphi_{tt})_{m-1}^n] \\ &+ \frac{k^3}{3!}[(\varphi_{ttt})_{m+1}^n - (\varphi_{ttt})_{m-1}^n] + O(k^4) \end{aligned}$$

Dividiendo por $2hk$:

$$\implies \frac{\varphi_{m+1}^{n+1} - \varphi_{m-1}^{n+1} - \varphi_{m+1}^n + \varphi_{m-1}^n}{2hk} = (\varphi_{tx})_m^n + \frac{h^2}{3!}(\varphi_{txxx})_m^n + \frac{k^2}{4hk}[(\varphi_{tt})_{m+1}^n - (\varphi_{tt})_{m-1}^n] + \frac{k^3}{3! \cdot 2hk}[(\varphi_{ttt})_{m+1}^n - (\varphi_{ttt})_{m-1}^n] + \frac{O(k^4)}{2hk}$$

$$\implies \frac{\varphi_{m+1}^{n+1} - \varphi_{m-1}^{n+1} - \varphi_{m+1}^n + \varphi_{m-1}^n}{2hk} = (\varphi_{tx})_m^n + \frac{h^2}{3!}(\varphi_{txxx})_m^n + \frac{k}{4h}[(\varphi_{tt})_{m+1}^n - (\varphi_{tt})_{m-1}^n] + \frac{k^2}{3! \cdot 2h}[(\varphi_{ttt})_{m+1}^n - (\varphi_{ttt})_{m-1}^n] + \frac{O(k^3)}{2h} + O(h^4)$$

$$\therefore (\varphi_{tx})_m^n - \frac{\varphi_{m+1}^{n+1} - \varphi_{m-1}^{n+1} - \varphi_{m+1}^n + \varphi_{m-1}^n}{2hk} = \left(\frac{h^2}{3!}(\varphi_{txxx})_m^n + \frac{k}{4h}[(\varphi_{tt})_{m+1}^n - (\varphi_{tt})_{m-1}^n] \right) + \frac{O(k^2)}{2h} + O(h^4)$$

Vemos que:

$$(P\varphi)_m^n - P_{kh}\varphi_m^n$$

$$\implies [(\varphi_t)_m^n - (\frac{\varphi_{m+1}^{n+1} - \varphi_m^n}{k})] + c[(\varphi_{tx})_m^n - (\frac{\varphi_{m+1}^{n+1} - \varphi_{m-1}^{n+1} - \varphi_{m+1}^n + \varphi_{m-1}^n}{2kh})] + a[(\varphi_x)_m^n - (\frac{\varphi_{m+1}^n - \varphi_{m-1}^n}{2h})]$$

Lo que nos queda:

$$\begin{aligned} \implies \frac{k}{2}(\varphi_{tt})_m^n + O(k^2) + c(\frac{h^2}{3!}(\varphi_{txxx})_m^n + \frac{k}{4h}[(\varphi_{tt})_{m+1}^n - (\varphi_{tt})_{m-1}^n] + \frac{O(k^2)}{2h} + O(h^4)) + a(\frac{h^2}{3!}(\varphi_{xxx})_m^n + O(h^4)) \\ \implies \frac{k}{2}(\varphi_{tt})_m^n + \frac{ch^2}{3!}(\varphi_{txxx})_m^n + \frac{ck}{4h}[(\varphi_{tt})_{m+1}^n - (\varphi_{tt})_{m-1}^n] + \frac{ah^2}{3!}(\varphi_{xxx})_m^n + O(h^4) + \frac{O(k^2)}{2h} \end{aligned}$$

Vemos que si el $\lim_{h,k \rightarrow 0} \frac{k}{h} = 0$ entonces el esquema es consistente

✓ Ejercicio 3. $\frac{23}{30}$



✓ a. $\frac{8}{15}$

Si un problema de valores iniciales está "bien puesto", este debe de cumplir que:

$$\forall T > 0; \exists C_T > 0 \text{ tal que}$$

$$\int_{-\infty}^{\infty} |u(t, x)|^2 dx \leq C_T \int_{-\infty}^{\infty} |u(0, x)|^2 dx \quad \forall 0 \leq t \leq T$$

Tenemos que la solución para la E.D.P de primer orden para $Pu = 0$ donde $Pu = u_t + au_x + bu$ está dada por:

$$u(t, x) = u_0(x - at)e^{-bt} + \int_0^t f(s, x - a(t - s))e^{-b(t-s)} ds \quad \text{Subra}$$

Si u_0 y f son funciones continuas y acotadas la solución $u(t, x)$ es continua en (t, x) y diferenciable en t y x , por lo tanto la solución existe. Sustituyendo la solución $u(t, x)$ obtenemos que:

$$\int_{-\infty}^{\infty} |u(t, x)|^2 dx = \int_{-\infty}^{\infty} |u_0(x - at)e^{-bt} + \int_0^t f(s, x - a(t - s))e^{-b(t-s)} ds|^2 dx$$

Tenemos que $|a + b|^2 \leq |a|^2 + |b|^2$, con lo que:

$$\int_{-\infty}^{\infty} |u(t, x)|^2 dx \leq \int_{-\infty}^{\infty} |u_0(x - at)e^{-bt}|^2 dx + \int_{-\infty}^{\infty} |\int_0^t f(s, x - a(t - s))e^{-b(t-s)} ds|^2 dx \quad \text{Subra}$$

Como $f(t, x) = 0$, tenemos que:

$$\begin{aligned} \int_{-\infty}^{\infty} |u(t, x)|^2 dx &\leq \int_{-\infty}^{\infty} |u_0(x - at)e^{-bt}|^2 dx \\ &= e^{-2bt} \int_{-\infty}^{\infty} |u_0(x - at)|^2 dx \end{aligned}$$

Haciendo el cambio de variable $\xi = x - at$, tenemos que $d\xi = dx$. Y los limites de integración no cambian, obtenemos que:

$$\int_{-\infty}^{\infty} |u(t, x)|^2 dx \leq e^{-2bt} \int_{-\infty}^{\infty} |u_0(\xi)|^2 d\xi = e^{-2bt} \int_{-\infty}^{\infty} |u_0(x)|^2 dx$$

Con $C_T = e^{-2bt}$ la cual es una constante que no diverge para valores de $t > 0$ y $b > 0$, por lo tanto el valor de la constante C_T va a ser

7 ≤ 0 . *Cart depende de t, debes acotar*

Podemos concluir que el problema de valores iniciales para la ecuación de primer orden $Pu = 0$ está "bien propuesto".

✓ b. $\frac{15}{16}$

Aplicando el siguiente corolario:

- Si un esquema de diferencias finitas (D.F) de **un paso** con coeficientes constantes se modifica y la modificación genera un factor de amplificación unicamente un cambio que es $O(k) \forall \xi$, entonces el esquema es estable si solo si el esquema original lo es.

Vemos que el factor de amplificación de nuestro esquema se modifica por un $O(k)$:

$$\begin{aligned} \frac{g^{n+1}e^{im\theta} - g^n e^{im\theta}}{k} + \frac{a}{2h}(g^{n+1}e^{i(m+1)\theta} - g^{n+1}e^{i(m-1)\theta}) + \frac{b}{2}(g^n e^{i(m+1)\theta} + g^n e^{i(m-1)\theta}) &= 0 \\ \Rightarrow g^n e^{im\theta}[(g - 1) + \frac{a\lambda}{2}(g(e^{i\theta} - e^{-i\theta}) + \frac{kb}{2}(e^{i\theta} + e^{-i\theta})) &= 0 \end{aligned}$$

Organizando el factor de amplificación y despejando g :

$$\begin{aligned} g(1 + ia\lambda \sin \theta) - 1 + bk \cos \theta &= 0 \\ \Rightarrow g &= \frac{1 - bk \cos \theta}{1 + ia\lambda \sin \theta} \\ \Rightarrow g &= \frac{1}{1 + ia\lambda \sin \theta} - \frac{bk \cos \theta}{1 + ia\lambda \sin \theta} \\ \Rightarrow g &= \frac{1}{1 + ia\lambda \sin \theta} - k \frac{b \cos \theta}{1 + ia\lambda \sin \theta} \\ \Rightarrow g &= \frac{1}{1 + ia\lambda \sin \theta} - O(k) \end{aligned}$$

Tenemos que si el esquema:

$$(*) \frac{V_m^{n+1} - V_m^n}{k} + \frac{a}{2h}(V_{m+1}^{n+1} - V_{m-1}^{n+1})$$

Es estable nuestro esquema tambien va a ser estable. Encontrando el factor de amplificación de $(*)$

$$\begin{aligned} \Rightarrow \frac{g^{n+1}e^{im\theta} - g^n e^{im\theta}}{k} + \frac{a}{2h}(g^{n+1}e^{i(m+1)\theta} - g^{n+1}e^{i(m-1)\theta}) &= 0 \\ \Rightarrow g^n e^{im\theta}[g - 1 + \frac{a\lambda}{2}g(e^{i\theta} - e^{-i\theta})] &= 0 \\ \Rightarrow g^n e^{im\theta}[g - 1 + \frac{a\lambda}{2}g(2i \sin \theta)] &= 0 \end{aligned}$$

Lo que nos da:

$$\begin{aligned} \Rightarrow g - 1 + \frac{a\lambda}{2}g(2i \sin \theta) &= 0 \\ \Rightarrow g(1 + a\lambda i \sin \theta) - 1 &= 0 \end{aligned}$$

$$\implies g(1 + a\lambda i \sin \theta) - 1 = 0$$

$$\implies g = \frac{1}{1 + a\lambda i \sin \theta}$$

$$\implies |g|^2 = \frac{1}{|1 + a\lambda i \sin \theta|^2}$$

$$\implies |g|^2 = \frac{1}{1 + (a\lambda)^2 \sin^2 \theta}$$

Se debe de cumplir que $|g|^2 \leq 1$ y como $0 \leq \sin^2 \theta \leq 1$ y $(a\lambda)^2$ siempre va a ser un numero positivo, tenemos que el factor de amplificación del esquema (*) va a ser menor o igual a uno. Dando como resultado que el esquema (*) siempre va a ser estable. En consecuencia nuestro esquema va a ser estable.

Analizando la consistencia del esquema vemos que:

- $(\varphi_t)_m^n - \frac{(\varphi)_{m+1}^{n+1} - (\varphi)_m^n}{k} = \frac{k}{2}(\varphi_{tt})_m^n + O(k^2)$
- $(\varphi_x)_m^n - \frac{(\varphi)_{m+1}^{n+1} - (\varphi)_{m-1}^{n+1}}{2h} = k(\varphi_{xt})_m^n + \frac{k^2}{2}(\varphi_{xtt})_m^n + \frac{h^2}{3!}(\varphi_{xxx})_m^{n+1} + O(h^4) + O(k^3)$
- $(\varphi)_m^n - \frac{(\varphi)_{m+1}^n + (\varphi)_{m-1}^n}{2} = \frac{h^2}{2}(\varphi_{xx})_m^n + O(h^3)$

De lo anterior podemos ver que para cualquier función "suave" $\varphi(t, x)$ se cumple:

$$(P\varphi)_m^n - P_{kh}\varphi_m^n \longrightarrow 0 \text{ cuando } h, k \longrightarrow 0$$

Por lo tanto el esquema propuesto es consistente.

Como el problema de valores iniciales está "bien puesto" y el esquema propuesto es consistente y estable se puede concluir por teorema que el esquema es convergente.